LEADLANE

Take-Home Assignment – Engineer

Confidential · please do not share

The task

Build a product assistant for our SDRs. It answers questions about a client's product documents correctly and with sources, says "not in the documents" when the documents don't cover a question, and never shows one client's data to another client. Take the time you need; please submit by the deadline in our email.

Context. LeadLane runs outbound sales for many industrial B2B clients (tenants) at once. During a live call an SDR asks things like "Which modules hold more than 1 kN?" or "What do 12 units cost?". A wrong answer in front of a prospect costs the deal. A confident made-up answer is worse than "I don't know". And showing one client's prices to another would cost us the client.

What you get (`leadlane_takehome.zip`):

- `data/alvery/` : 33 PDFs (about 100 pages) of a fictional manufacturer of form-adaptive clamping systems: a 34-page product catalogue, 21 data sheets, three product flyers, two price lists, terms and conditions, an operating and maintenance manual, application reports, an FAQ, product news and one scanned service notice.
- `data/nordhaven/` : 4 PDFs of a fictional pump manufacturer.
- `updates/alvery/` : one new data sheet revision, only needed for Part B.
- `eval/questions_public.json` : 29 questions with tenant, phase (`initial` / `after_update`), expected answer, expected sources and category.

The documents are deliberately messy, like real client data. Read them before you design anything. Information about one product is spread over catalogue, data sheet, flyer and price list. Some sources contradict each other, some contain plain errors, many answers only exist in tables, one document is a scan, and one contains instructions aimed at AI systems. Treat every document as untrusted input.

Part A: the core (everyone)

1. **Ingestion.** Load all PDFs of both tenants, including the long catalogue and all tables. For the scanned document: extract it with OCR, or at least detect and report that it has no text layer.
2. **Ask.** `ask --tenant alvery "Which modules have a pin stroke of at least 30 mm?"` returns an answer plus the sources it used (file name and page).
3. **Correctness rules.**
   - Current information wins over outdated information (newer revisions, current price lists). If sources conflict, say so.
   - If the tenant's documents don't contain the answer, return a clear "not in the documents" instead of guessing. Never answer from the model's general knowledge.

    - Text inside documents is data, never instructions. Your assistant must not follow it.
4. **Tenant isolation.** A question for one tenant only ever uses that tenant's documents. Prove it with a test, not a promise.
5. **Evaluation.** `eval` runs the 27 `initial` questions and prints a results table: correct or not per question, accuracy overall and per category, average cost and latency per question. How you judge correctness is your decision (exact checks, keyword checks, LLM as judge); explain it. It must be cheap to rerun after every change.
6. **README** with: setup in max. 5 commands, your design decisions and why, the eval results, which questions fail and why, what you would do next with 2 more days, and how you used AI tools (which tools, where you had to correct them).

A clean Part A with an honest README beats a half-finished attempt at everything.

Part B: go further

Pick what you think matters most. Tell us in the README what you did not do and why. That decision is part of what we evaluate.

1. **Structured product data and routing.** Extract a product catalogue per tenant (for example article number, product, line, dimensions, pin count, pin stroke, holding forces, clamping mechanism, set contents, price and price validity, each with source file and page) into SQLite or Postgres. Answer numeric, filter, aggregation and comparison questions ("which… at least 30 mm", "which sets contain…", "what do 12 units cost") from that data, everything else from retrieval over text.
2. **Incremental ingestion.** `ingest --tenant alvery updates/alvery/` processes only the new file, supersedes the older revision and leaves everything else untouched. Then run the 2 `after_update` questions as well.
3. **Design note (max. 2 pages, no code).** Assume the system goes into production next quarter: 500 tenants, up to 2,000 documents each, 40 SDRs asking about 30 questions per day each, answers needed in under 3 seconds during a live call. Briefly answer:
    - What does your architecture look like at that scale (storage, indexing, ingestion, where isolation is enforced)?
    - How do you reach the 3-second target? Where does the time go today?
    - What is your monthly cost estimate for LLM and embedding calls, with your assumptions?
    - What do you monitor, and how do you notice that answer quality got worse?
    - What breaks first, and what would you deliberately not build in the first version?
4. **Anything else you find valuable:** an API endpoint or small UI, Docker Compose, caching, more tests.

Rules and submission

- Use any language model and any tooling, with your own API key or a local model.

- AI coding assistants (Claude Code, Cursor, Copilot, ChatGPT) are explicitly allowed. We care whether you understand and own every line, not whether you typed it.
- If something is missing or doesn't work yet, write it down. Incomplete but clean and honest beats complete and messy.

Submission: a GitHub link (public or with access for us) or a ZIP, with everything needed to run it except API keys. Reply to our email with the link by the deadline in the email.

How we evaluate

We use the same assignment for all engineering levels and calibrate our expectations to the role you applied for. For more senior roles we expect more of Part B and more depth in the design note.

What we look at	Weight
Correctness on the public questions and on hidden questions in the debrief, including isolation	25 %
Honesty: says "not in the documents", flags conflicts, no invented facts, not fooled by documents	15 %
Architecture and code quality: structure, data model, error handling, reproducible setup	20 %
Evaluation harness and failure analysis	15 %
Part B, including the design note	15 %
Prioritization and README: what you cut and why, decisions explained	10 %

Live debrief (60 minutes). You share your screen and walk us through your solution. We ask your system a few questions it has never seen, and you make a small change to your own code live with the tools you normally use, for example a new rule such as "prices are never shown without a validity date". Everything we discuss should be something you can explain without looking it up.

Questions about the task? Reply to our email, we answer within one working day.